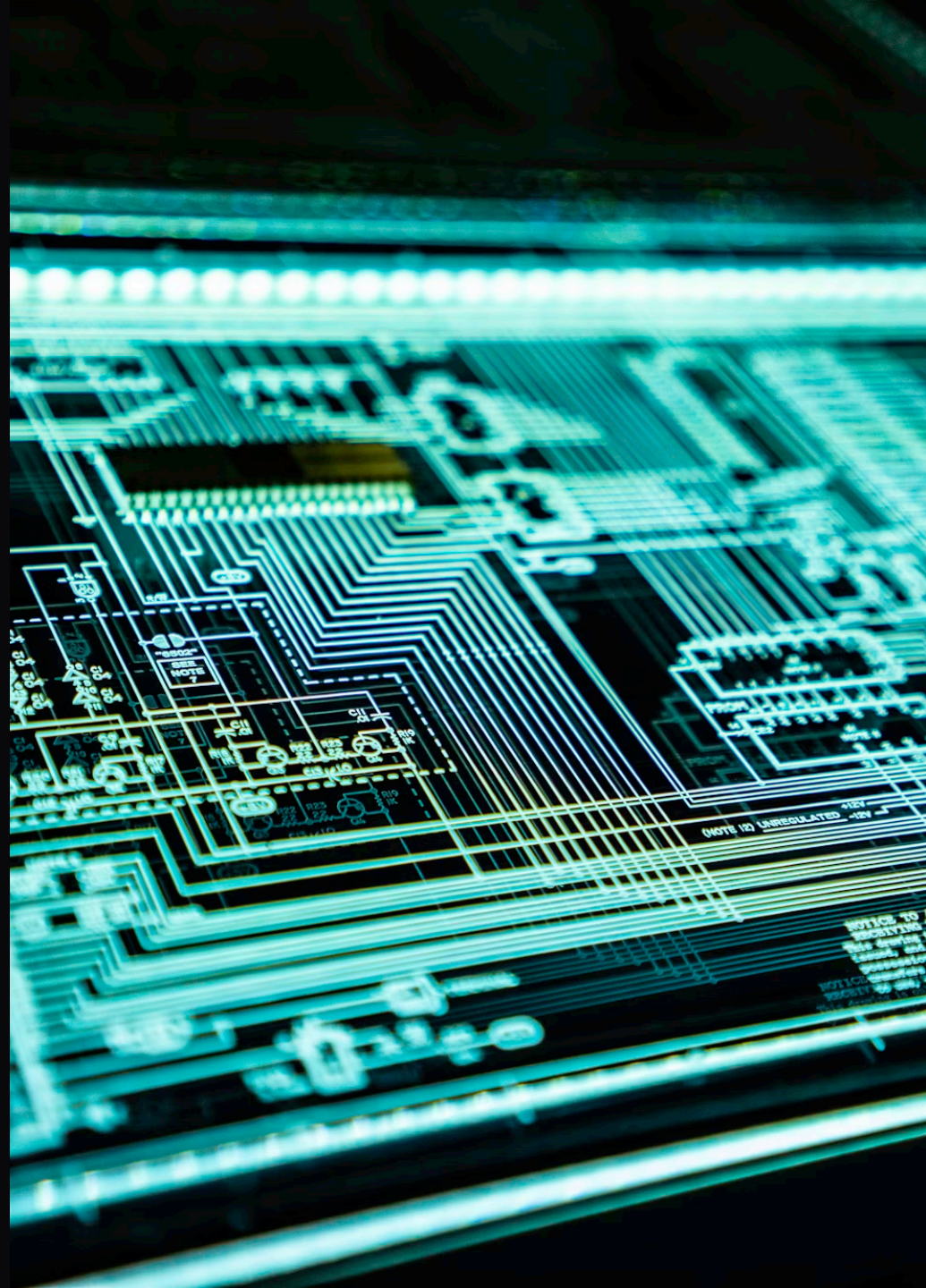
L13: CREATING MODULES

MODULE 3: ARCHITECTURE



MISSION BRIEFING

> "You've used the tools provided by the system. Now, it's time to forge your own."
> — Emily Chen, Handler

We are moving from **consumers** of code to **creators** of libraries.

THE PROBLEM: MONOLITHS

Writing all your code in one file is like building a skyscraper in a single room.

- Hard to navigate
- Hard to debug
- Impossible to reuse

Solution: Modular Architecture.

WHAT IS A MODULE?

In Python, **every file ending in `.py` is a module.**

Yes, you've been creating modules since Lesson 1.
You just haven't imported them yet.

THE FILE SYSTEM

```
project/
├── main.py       <-- The Entry Point
├── utils.py      <-- A Module
└── config.py     <-- Another Module
```

Cole: "Think of `main.py` as your command center, and other files as your weapon racks."

CREATING YOUR FIRST MODULE

`tools.py` :

```
def hack_terminal():
    print("Accessing...")

def secure_line():
    print("Encrypting...")
```

`main.py` :

```
import tools

tools.hack_terminal()
```

THE IMPORT PROCESS

When you type `import tools`, Python does three things:

1. **Finds** the file `tools.py`.
2. **Executes** the entire file from top to bottom.
3. **Creates** a module object named `tools`.

DEEP CS: THE MODULE OBJECT

Modules are **objects** in memory.

Module: 'tools'

.hack_terminal() .secure_line()

SYNTAX VARIANT 1: IMPORT WHOLE

```
import tools

tools.hack_terminal()
```

Pros: Distinct namespace. No name collisions.

Cons: More typing.

SYNTAX VARIANT 2: FROM ... IMPORT

```
from tools import hack_terminal

hack_terminal()  # No prefix needed!
```

Pros: Concise.

Cons: Can overwrite existing functions with the same name.

SYNTAX VARIANT 3: ALIASING

```
import tools as t
from config import super_long_variable_name as conf

t.hack_terminal()
print(conf)
```

Cole: "Use this for common libraries like `import pandas as pd` or `import numpy as np`."

THE `__pycache__` GHOST

Ever seen this folder appear?

`__pycache__/tools.cpython-310.pyc`

This is **Compiled Bytecode**. Python compiles your module to speed up loading next time. It's safe to ignore (and add to `.gitignore`).

EXECUTION FLOW DEMO

`squad.py` :

```
print("Squad module loading...")  # Runs on import!

def recruit():
    print("Recruiting...")
```

`main.py` :

```
import squad
print("Main starting...")
```

Output:

VARIABLE SCOPE IN MODULES

Variables defined in a module are **Global** to that module, but specific to its **Namespace.**

`config.py` :

```
API_KEY = "12345"
```

`main.py` :

```
import config
print(config.API_KEY) # Safe
print(API_KEY)        # Error! Not in main's scope
```

THE SPECIAL VARIABLE: `__name__`

Every module has a hidden variable called `__name__`.

- If run directly: `__name__ == "__main__"`
- If imported: `__name__ == "filename"`

THE GUARD CLAUSE

Emily: "This is the most critical pattern in Python scripting."

```
def routine():
    print("Running routine")

if __name__ == "__main__":
    print("Running directly as a script")
    routine()
```

This prevents code from running when you just want to import the functions.

MODES OF OPERATION

Mode	`__name__`	Use Case
Script	`"__main__"`	Running the app (`python app.py`)
Module	`"tools"`	Providing utilities (`import tools`)

CIRCULAR IMPORTS: THE DEADLOCK

`a.py` imports `b`
`b.py` imports `a`

Result: **ImportError**.

Why? Module A needs B to finish loading, but B needs A to finish loading.
Fix: Architect your code to avoid cycles. Move common code to a third module `c.py`.

STANDARD LIBRARY VS CUSTOM

You can mix them seamlessly.

```
import random       # Standard Lib
import time         # Standard Lib
import my_engine    # Custom Module
```

Python looks in:

1. Current Directory
2. PYTHONPATH (System paths)
3. Standard Library paths

ORGANIZING LARGER PROJECTS

When you have too many modules, you need a **Package.**

```
cyber_security/
├── __init__.py    <-- Marks folder as package
├── network.py
├── crypto.py
└── auth.py
```

Usage: `from cyber_security import crypto`

THE `__init__.py` FILE

Often empty, but it tells Python: "Treat this directory as a package."

It can also be used to expose specific functions to the outside world, simplifying imports.

PRO TIP: DOCSTRINGS

Document your modules!

`tools.py`:

```
"""
TOOLS MODULE
Contains utilities for system interaction.
Author: Agent 42
"""

def scan():
    """Scans the local network."""
    pass
```

EXERCISE: THE TOOLKIT

1. Create `math_core.py`.
2. Add functions: `add(a, b)`, `power(a, b)`.
3. Create `main.py`.
4. Import `math_core` and use the functions.

COMMON ERROR:

`ModuleNotFoundError`

```
ModuleNotFoundError: No module named 'tols'
```

1. Check spelling.
2. Is the file in the **same folder**?
3. Did you forget the `.py` extension when creating logic (but don't add it in import)?

DEBUGGING MODULES

If you change a module code, you must **restart** the main script to see changes.

Python keeps the *old* version in memory for efficiency.

Cole: "Reloading modules dynamically is possible but advanced. For now, just restart."

BEST PRACTICES

1. **Lowercase filenames:** `my_module.py` (Snake Case).
2. **One purpose:** Don't mix database code with UI code.
3. **No Side Effects:** Importing a module shouldn't print things or launch missiles. It should just define definitions.

SUMMARY

- **Modules** are `.py` files.
- **Import** brings them into your namespace.
- **Namespaces** prevent variable clashes.
- `if __name__ == "__main__":` controls execution.
- **Packages** organize modules into folders.

NEXT MISSION

You have the blueprints. Now we build the framework.

Prepare for **Turtle Graphics**. We are moving from text to visuals.

> "The grid is loading..."